

CI/CD Pipeline with GitHub Actions & Docker (No Cloud Needed)

Introduction

Continuous Integration and Continuous Deployment (CI/CD) pipelines are essential for automating the software delivery process. This project demonstrates a local CI/CD setup using GitHub Actions and Docker to automate building, testing, and deploying applications without relying on cloud services. The implementation focuses on streamlining the development workflow and ensuring consistent deployments.

Abstract

The CI/CD Pipeline project integrates GitHub Actions with Docker to automate application build and deployment workflows. When a developer pushes code to the repository, GitHub Actions automatically runs unit tests, builds a Docker image, and pushes it to Docker Hub. The final image is then deployed locally using Minikube or a virtual machine, simulating a production environment. This setup demonstrates a complete CI/CD lifecycle without requiring cloud infrastructure.

Tools Used

- GitHub Actions – For automating CI/CD workflows.
- Docker – For containerizing the application and ensuring environment consistency.
- Docker Hub – For storing and distributing container images.
- Minikube / Local VM – For running the application in a local deployment environment.
- YAML – For defining GitHub workflow configurations.

Steps Involved in Building the Project

- Created a sample application (e.g., Node.js or Python Flask app).
- Wrote a Dockerfile and docker-compose.yml for building and running the container locally.
- Configured a GitHub Actions workflow to automatically build the Docker image, run tests, and push the image to Docker Hub.
- Set up Docker Hub repository for image storage and version control.
- Used Minikube or a local VM to pull and run the latest Docker image automatically after each successful build.
- Verified the pipeline by making code changes and observing automatic builds and deployments.

Conclusion

This project successfully demonstrates how to implement a fully automated CI/CD pipeline using GitHub Actions and Docker without relying on any cloud platform. The approach ensures faster delivery cycles, improved reliability, and consistent deployments across environments. It serves as a practical foundation for learning DevOps automation and continuous delivery best practices.